ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Уговори, симулатори и поновљивост

Заједничко значење поруке и контролисан експеримент



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–18

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

19–26

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Разумети како уговор и поновљив симулатор омогућавају независну промену компоненти.

Порука без стабилног облика и значења тера примаоца да нагађа, а непоновљив сценарио не даје поуздан доказ поправке.

> НАПОМЕНА
> Уговори штите сарадњу компоненти.

01 · ПОЈМОВИ

Четири појма теме

Компоненте могу да сарађују само ако деле уговор о облику поруке и очекиваном понашању.

01 **Уговор**

облик и значење поруке

02 **Извор**

шаље према уговору

03 **Симулатор**

понавља познат сценарио

04 **Прималац**

валидира и тумачи

01 · ПОЈМОВИ

Уговор

Уговор описује облик поруке, њено значење и дозвољене вредности. Он спречава да прималац нагађа шта је извор хтео да пошаље.

01 Описује облик, значење и дозвољене вредности поруке.

02 Спречава нагађање о намери пошиљаоца.

03 Промена захтева верзију, не тиху измену.

Извор

Извор шаље поруку према договореном уговору и бира верзију коју прималац може да разуме. Слање није исто што и потврђен пословни исход.

01 Шаље према договореном уговору, не произвољно.

02 Бира верзију коју прималац разуме.

03 Слање ≠ потврђен пословни исход.

01 · ПОЈМОВИ

Симулатор

Симулатор намерно производи познат ток и сметње. Његова вредност је у томе што тим може исти сценарио да понови без спољне опреме.

01 Производи познат, поновљив ток и сметње.

02 Замењује реалну опрему контролисаним експериментом.

03 Не сме преузети пословну логику система.

01 · ПОЈМОВИ

Прималац

Прималац валидира поруку пре обраде и јасно одбија оно што не разуме. Тако непозната промена не постаје тиха грешка у пословном стању.

01 Валидира пре обраде, не после.

02 Јасно одбија оно што не разуме.

03 Непозната промена не сме тихо проћи у пословно стање.

Пример: прималац не разуме нову поруку

Пошиљалац проширује поруку новим пољем и сматра да промена нема последице.

01 Верзија говори примаоцу како да тумачи поруку.

02 Симулатор може поновити исти ток са старим и новим примаоцем.

03 Нејасна или непозната порука добија контролисан исход, а не произвољно тумачење.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема уговора између компоненти

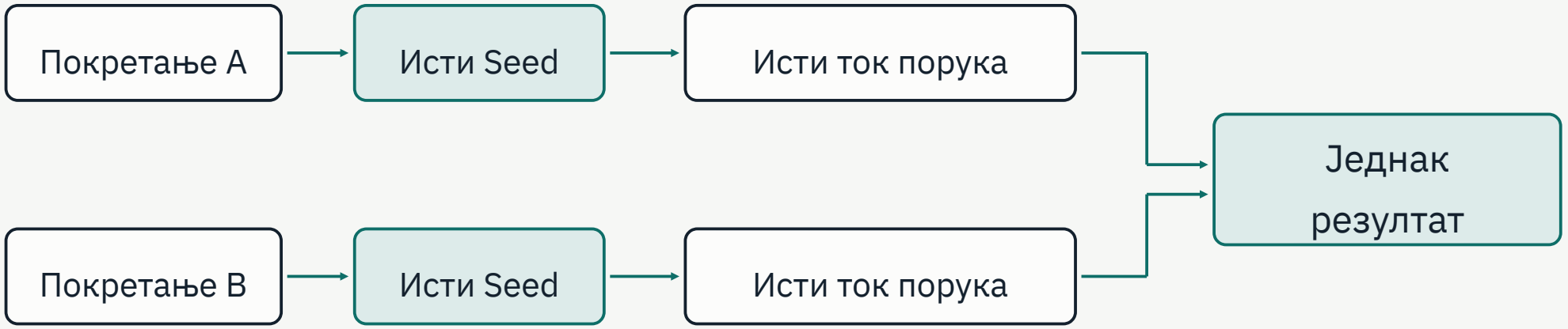
Стабилан уговор омогућава компонентама да се мењају без нагађања о значењу поруке.

Елемент	Улога у току
Пошиљалац	ствара поруку
Уговор	облик, значење и верзија
Симулатор	поновљив сценарио
Прималац	валидира и обрађује

02 · ДИЈАГРАМ

Понављив сценарио даје упоредив резултат

Исти Seed одређује исти улаз, па тест може да пореди два покретања.



Познат сценарио омогућава да се грешка понови после измене.

ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример показује две ствари које омогућавају да се компоненте развијају независно.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/02 Ugovori simulator"
dotnet run --project src/Odp.Vezba02.ConsoleUi
dotnet test Odp.Vezba02.sln
```

Правило у коду

Уговор се проверава на улазу, а свако одбијање има стабилан код.

C# · доменско правило

```
public Result Validate(TelemetryMessage message)
{
    ArgumentNullException.ThrowIfNull(message);
    if (message.ContractVersion != TelemetryContract.Version)
        return Result.Fail(ContractErrorCodes.UnsupportedVersion);
    if (string.IsNullOrWhiteSpace(message.StationId))
        return Result.Fail(ContractErrorCodes.StationIdRequired);
    if (message.Sequence <= 0)
        return Result.Fail(ContractErrorCodes.SequenceMustBePositive);
    if (message.SignalStrengthDbm is < TelemetryContract.MinSignalDbm or >
        TelemetryContract.MaxSignalDbm)
        return Result.Fail(ContractErrorCodes.SignalOutOfRange);
    return Result.Ok();
}
```

Тест који задаје квар

Исти сценарио (исти Seed) даје исте поруке, па се и грешка може поновити.

C# · NUnit

```
1   [Test]
2   public void Emit_WithTheSameScenario_ProducesTheSameMessages()
3   {
4       var first = new ScriptedStationSimulator(Scenario).Emit(20);
5       var second = new ScriptedStationSimulator(Scenario).Emit(20);
6       Assert.That(second, Is.EqualTo(first));
7   }
```

Испис демонстрације

Симулатор је adapter иза порта IStationSimulator, правила не знају да станица није права.

console · исходи демонстрације

```
Poruke simulatora:
  #1 09:00:00  -83.0 dBm
  #2 09:00:05 -105.0 dBm
  #3 09:00:10 -105.0 dBm
  #4 09:00:15    5.0 dBm
  #5 09:00:20 -104.0 dBm
  #6 09:00:25 -100.0 dBm
  #7 09:00:30  -81.0 dBm
  #8 09:00:35    5.0 dBm
Prihvaćeno: 6, odbijeno: 2
  #4 -> signal_out_of_range
  #8 -> signal_out_of_range
Ponovljeno pokretanje daje isti ishod: True
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Објављена верзија уговора чува очекивања постојећих корисника.
- Валидација одбија поруку коју систем не разуме, уместо да јој нагађа значење.
- Симулатор одваја стварни проблем система од случајности спољног света.

ЗАМКЕ

- Промена поруке без верзије и без размишљања о примаоцима.
- Симулатор који садржи пословну логику централног система.
- Тест који зависи од случајног времена или спољне услуге.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Промена поруке без верзије.

02 Симулатор преузима пословну логику система.

03 Тест зависи од случајног времена или спољне услуге.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Одредити мали скуп кључних порука.

02 Поновити исти сценарио са познатим улазом.

03 Одбити поруку коју компонента не разуме.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Која порука најпре треба да постане стабилан уговор и какав симулатор би проверио њено значење?

> ЗА ВЕЖБУ
>
> Коју поруку бисте најпре претворили у стабилан уговор и који сценарио симулатора би најбоље открио неспоразум између пошиљаоца и примаоца?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати у сценарио кашњење поруке (DelayEvery), тако да свака H-та порука носи MeasuredAt старији од претходне.

02 Увести правило уговора које такву поруку одбија новим кодом и покрити га тестом.

04 · ПРИМЕНА

Контролна тачка P1 (10.02.)

Прва контролна тачка (Вежба 1–2) проверава основне ентитете, поновљив симулатор и прве стабилне уговоре.

01 Mission/GroundStation/StationNode имплементирани са основним статусима и heartbeat провером.

02 Поновљив DeviceSimulator са најмање два профила понашања.

03 Најмање један pull request показује преглед diff-а.

04 · ПРИМЕНА

Резиме

Тим записује мали скуп порука и сценарија који представљају кључне токове.

Стабилан уговор омогућава компонентама да се мењају без нагађања о значењу поруке.

НАПОМЕНА

Наредна вежба: Идентитет, audit, observability и конфигурација.